SOP Generation Pipeline — Technical Documentation

Core Design Principle

The pipeline generates SOPs that match **whatever is in your Knowledge Base**.

There are no hardcoded formatting rules, no hardcoded document names, and no organisation-specific references anywhere in the codebase. Instead:

1. The **research agent** retrieves documents from your Bedrock KB.
2. It extracts their actual formatting conventions — section titles, table structures, numbering style, writing tone — into a `kb_format_context` object.
3. Every downstream agent (**content**, **formatter**, **QA**) receives `kb_format_context` in its prompt and mirrors those conventions.

Point the pipeline at a different KB, and the output format changes automatically.

Architecture

```
Caller (test/custom_sop.py)
      │
      ▼
  generate_sop()          ← src/graph/sop_workflow.py
      │
      ▼
  STATE_STORE             ← src/graph/state_store.py
  (keyed by workflow_id)
      │
      ▼
╔═══════════════════════════╗
║  Strands Graph            ║
║                           ║
║  [1. planning]            ║  → Outline: section titles from KB (or standard fallback)
║      ↓                    ║
║  [2. research]            ║  → KB retrieval + extract kb_format_context
║      ↓                    ║
║  [3. content]             ║  → Write each section using kb_format_context + KB facts
║      ↓                    ║
║  [4. formatter]           ║  → Render Markdown using kb_format_context
║      ↓                    ║
║  [5. qa]                  ║  → Score against kb_format_context
```

```
│      ↓        │
│ (revision loop  │
│ if not approved) │
│      └──────────────────→ [3. content]
└──────────────────────────────┘
      │
      ▼
  Final SOPState   ← formatted_markdown, qa_result, kb_format_context
```

Project Structure

```
sop_project/
├── src/
│   ├── graph/
│   │   ├── state_schema.py    # Pydantic models — includes kb_format_context
│   │   ├── state_store.py     # In-process shared state dict
│   │   └── sop_workflow.py    # Graph builder + async entry point
│   ├── agents/
│   │   ├── planning_agent.py  # Node 1: outline (uses kb_format_context if available)
│   │   ├── research_agent.py  # Node 2: KB retrieval + kb_format_context extraction
│   │   ├── content_agent.py   # Node 3: section writing (driven by kb_format_context)
│   │   ├── formatter_agent.py # Node 4: Markdown rendering (driven by kb_format_context)
│   │   └── qa_agent.py        # Node 5: quality check (evaluates against kb_format_context)
│   ├── prompts/
│   │   └── system_prompts.py  # All agent prompts — no hardcoded format rules
│   └── utils/
│       └── logger.py          # Logger factory
└── test/
    └── custom_sop.py          # Entry point — edit TOPIC / INDUSTRY / AUDIENCE
```

The kb_format_context Object

This is the key that makes the pipeline KB-agnostic. It is extracted by the research agent from actual KB document chunks and flows through every stage.

```
json
```

```
{
  "section_titles": [
    { "number": "1.0", "title": "<exact title from KB>" },
    ...
  ],
  "numbering_style": "1.0 / 2.0 / 6.3.1",
  "table_sections": [
    { "number": "3.0", "columns": ["ROLE", "RESPONSIBILITY"] },
    { "number": "4.0", "columns": ["TERM / ABBREVIATION", "DEFINITION"] },
    { "number": "8.0", "columns": ["Revision", "Effective Date", "Reason"] }
  ],
  "subsection_sections": ["2.0", "6.0", "7.0"],
  "prose_sections": ["1.0", "5.0"],
  "writing_style": "formal, imperative, third-person",
  "special_elements": ["indented numbered items for subsections"],
  "section_count": 8,
  "banned_elements": ["Method:", "Acceptance Criteria:", "Time Estimate:"]
}
```

If the KB returns no results, `kb_format_context` is null and all agents fall back to a generic standard SOP format — the pipeline never breaks.

File-by-File Reference

`src/graph/state_schema.py`

Defines all Pydantic models. Key additions:

Field	Where set	What it does
`kb_format_context`	`SOPState`	Top-level shortcut to KB format conventions
`kb_format_context`	`ResearchFindings`	Where research agent writes it first

`src/graph/state_store.py`

Module-level dict — holds live SOPState objects keyed by `workflow_id`. The Strands Graph only passes strings between nodes; this is the side-channel for rich shared data.

`src/prompts/system_prompts.py`

All five agent prompts in one place. Key design:

- No document names, organisation names, or hardcoded section lists
- Every prompt tells the agent to read `KB FORMAT CONTEXT` from the user message
- Fallback instructions for when KB context is absent

`src/agents/planning_agent.py`

Node 1. Calls Bedrock Converse with Structured Outputs (JSON Schema mode). Uses `kb_format_context` from state if available; otherwise falls back to a standard 8-section SOP structure. Schema allows 1–20 sections.

`src/agents/research_agent.py`

Node 2. The most complex agent. Responsibilities:

1. **Multi-round KB retrieval** (up to 3 rounds, all queries fully dynamic):
   - Round 1: topic + industry + outline-title queries
   - Round 2: keyword-only shortened queries
   - Round 3: generic structural SOP terms
2. **LLM synthesis**: Claude reads KB chunks and produces:
   - `section_insights` — KB facts per section number
   - `kb_format_context` — formatting conventions observed in the KB
3. **State propagation**: copies `kb_format_context` to `SOPState` top-level

Environment controls:

Variable	Default	Meaning
`KNOWLEDGE_BASE_ID`	*required*	Your Bedrock KB ID
`RESEARCH_REQUIRE_KB`	`1`	Fail if KB returns 0 hits
`RESEARCH_FALLBACK`	`0`	Use generic findings if KB fails
`KB_MIN_HITS`	`1`	Minimum acceptable hits
`KB_FORCE_HITS`	`1`	Retry rounds until min hits met

`src/agents/content_agent.py`

Node 3. Writes all SOP sections sequentially. Each section call receives:

- `KB FORMAT CONTEXT` — tells the agent which schema to use for this section type

- KB Insights — actual facts from KB for this specific section
- Outline subsection titles

src/agents/formatter_agent.py

Node 4. Passes the complete content JSON + kb_format_context to Claude, which renders Markdown following the KB's actual structure — tables where the KB uses tables, indented subsections where the KB uses them, etc.

src/agents/qa_agent.py

Node 5. Evaluates the document against kb_format_context. If context is missing, falls back to general SOP quality checks. Revision loop: up to 2 retries if score < 8.0.

Execution Flow

```
1. Edit TOPIC / INDUSTRY / AUDIENCE in test/custom_sop.py
2. Run: python test/custom_sop.py

3. generate_sop() creates SOPState → stored in STATE_STORE

4. planning_agent:
   - Reads kb_format_context from state (null on first run)
   - Falls back to standard 8-section structure
   - Produces SOPOutline with topic-appropriate subsection titles

5. research_agent:
   - Builds dynamic queries from topic + industry + outline titles
   - Runs multi-round concurrent KB retrieval
   - Calls Claude to synthesise:
       * section_insights (KB facts per section)
       * kb_format_context (formatting conventions from KB)
   - Writes kb_format_context to SOPState

6. content_agent:
   - Iterates all sections
   - Per section: sends KB FORMAT CONTEXT + KB facts → Claude writes JSON
   - Stores content_sections dict in SOPState

7. formatter_agent:
   - Sends content_sections + kb_format_context → Claude renders Markdown
   - Stores formatted_markdown in SOPState

8. qa_agent:
```

```
   - Sends formatted_markdown + kb_format_context → Claude scores 5 criteria
   - If approved: graph exits
   - If not approved and retries < 2: loop back to content_agent

9. Final SOPState returned → .md / .docx / .pdf written to disk
```

Quick Start

```bash
# 1. Set required environment variables
export KNOWLEDGE_BASE_ID="your_kb_id_here"
export AWS_REGION="us-east-2"
# Optionally override models:
# export MODEL_PLANNING="arn:aws:bedrock:..."
# export MODEL_RESEARCH="arn:aws:bedrock:..."

# 2. Install dependencies
pip install strands-agents boto3 pydantic python-docx reportlab

# 3. Edit test/custom_sop.py — set TOPIC, INDUSTRY, AUDIENCE

# 4. Run
python test/custom_sop.py
```

Environment Variables

Variable	Required	Default	Purpose
`KNOWLEDGE_BASE_ID`	**Yes**	—	Your Bedrock KB ID
`AWS_REGION`	No	`us-east-2`	AWS region
`MODEL_PLANNING`	No	claude-sonnet-4-6	Bedrock model for planning
`MODEL_RESEARCH`	No	claude-sonnet-4-6	Bedrock model for research
`MODEL_CONTENT`	No	claude-sonnet-4-6	Bedrock model for content
`MODEL_FORMATTER`	No	claude-sonnet-4-6	Bedrock model for formatter

Variable	Required	Default	Purpose
`MODEL_QA`	No	claude-sonnet-4-6	Bedrock model for QA
`KB_MAX_RESULTS`	No	`20`	KB hits per query
`KB_MIN_HITS`	No	`1`	Min total KB hits required
`KB_MIN_SCORE`	No	`0.0`	Min relevance score to keep
`RESEARCH_REQUIRE_KB`	No	`1`	Fail if KB returns 0 hits
`RESEARCH_FALLBACK`	No	`0`	Use generic findings if KB fails
`KB_FORCE_HITS`	No	`1`	Retry until min hits met
`KB_MAX_ROUNDS`	No	`3`	Max retrieval rounds
`LOG_LEVEL`	No	`INFO`	Logging verbosity

Troubleshooting

KB returning 0 hits:

- Verify `KNOWLEDGE_BASE_ID` is correct and your AWS credentials have `bedrock:Retrieve` permission
- Set `LOG_LEVEL=DEBUG` to see all query strings and hit counts per query
- Set `RESEARCH_REQUIRE_KB=0` + `RESEARCH_FALLBACK=1` to bypass KB during development

kb_format_context is null after research:

- KB chunks may not contain enough structural cues (heading patterns, table rows, etc.)
- Try lowering `KB_MIN_SCORE` to include more chunks
- The pipeline continues with generic formatting — this is not a fatal error

Format doesn't match KB:

- Increase `KB_MAX_RESULTS` so more KB chunks reach the synthesis step
- Check that KB chunks contain actual document text (not just metadata)
- Review what `kb_format_context` was populated with: add `LOG_LEVEL=DEBUG` and look for the "KB format context extracted" log line

Graph not routing:

- Check that `workflow_id::` appears in all agent return strings
- Verify `STATE_STORE[workflow_id]` is populated before `invoke_async`